UCS301 Data Structures

Lab Assignment 5: Single Linked List (Week 5)

Name: Nandini Sharma
Roll No: 2026010076
Group: MCA3

Question 1: Menu Driven Program on a Singly Linked List

Insertion (beginning, end, before/after a node), deletion (beginning, end, specific node), search with position, display

Program:

```
/* Menu driven program on Singly Linked List */
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};
struct node *head = NULL;

struct node* newNode(int x) {
    struct node *p = (struct node*)malloc(sizeof(struct node));
    p->data = x;
    p->next = NULL;
    return p;
}

void insertBegin(int x) {
    struct node *p = newNode(x);
    p->next = head;
    head = p;
    printf("%d inserted at beginning\n", x);
}

void insertEnd(int x) {
    struct node *p = newNode(x), *t;
    if (head == NULL) head = p;
    else {
        t = head;
        while (t->next != NULL) t = t->next;
        t->next = p;
    }
    printf("%d inserted at end\n", x);
}

void insertAfter(int val, int x) {
    struct node *t = head, *p;
    while (t != NULL && t->data != val) t = t->next;
    if (t == NULL) { printf("Node %d not found\n", val); return; }
    p = newNode(x);
    p->next = t->next;
    t->next = p;
    printf("%d inserted after %d\n", x, val);
}

void insertBefore(int val, int x) {
    struct node *t = head, *p;
    if (head == NULL) { printf("List is empty\n"); return; }
    if (head->data == val) { insertBegin(x); return; }
    while (t->next != NULL && t->next->data != val) t = t->next;
    if (t->next == NULL) { printf("Node %d not found\n", val); return; }
```

```
    p = newNode(x);
    p->next = t->next;
    t->next = p;
    printf("%d inserted before %d\n", x, val);
}

void deleteBegin() {
    struct node *t = head;
    if (head == NULL) { printf("List is empty\n"); return; }
    head = head->next;
    printf("%d deleted from beginning\n", t->data);
    free(t);
}

void deleteEnd() {
    struct node *t = head, *prev = NULL;
    if (head == NULL) { printf("List is empty\n"); return; }
    while (t->next != NULL) { prev = t; t = t->next; }
    if (prev == NULL) head = NULL;          /* only one node */
    else prev->next = NULL;
    printf("%d deleted from end\n", t->data);
    free(t);
}

void deleteValue(int val) {
    struct node *t = head, *prev = NULL;
    while (t != NULL && t->data != val) { prev = t; t = t->next; }
    if (t == NULL) { printf("Node %d not found\n", val); return; }
    if (prev == NULL) head = t->next;       /* first node */
    else prev->next = t->next;
    printf("Node %d deleted\n", val);
    free(t);
}

void search(int val) {
    struct node *t = head;
    int pos = 1;
    while (t != NULL) {
        if (t->data == val) { printf("Node %d found at position %d\n", val, pos);
return; }
        t = t->next;
        pos++;
    }
    printf("Node %d not found\n", val);
}

void display() {
    struct node *t = head;
    if (head == NULL) { printf("List is empty\n"); return; }
    printf("List: ");
    while (t != NULL) { printf("%d->", t->data); t = t->next; }
    printf("NULL\n");
}

int main() {
    int ch, x, v;
    do {
        printf("\n1.Insert at beginning  2.Insert at end  3.Insert before a node\n");
        printf("4.Insert after a node  5.Delete from beginning  6.Delete from end\n");
        printf("7.Delete a node  8.Search  9.Display  10.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &ch);
        if (ch == 1) { printf("Enter value: "); scanf("%d", &x); insertBegin(x); }
        else if (ch == 2) { printf("Enter value: "); scanf("%d", &x); insertEnd(x); }
        else if (ch == 3) { printf("Insert new value: "); scanf("%d", &x);
                            printf("Before node: "); scanf("%d", &v); insertBefore(v,
x); }
        else if (ch == 4) { printf("Insert new value: "); scanf("%d", &x);

        printf("After node: "); scanf("%d", &v); insertAfter(v, x); }
```

```
        else if (ch == 5) deleteBegin();
        else if (ch == 6) deleteEnd();
        else if (ch == 7) { printf("Enter node to delete: "); scanf("%d", &x);
deleteValue(x); }
        else if (ch == 8) { printf("Enter node to search: "); scanf("%d", &x);
search(x); }
        else if (ch == 9) display();
    } while (ch != 10);
    return 0;
}
```

Output:

```
1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 9
List is empty

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 1
Enter value: 30
30 inserted at beginning

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 2
Enter value: 60
60 inserted at end

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 1
Enter value: 10
10 inserted at beginning

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 2
Enter value: 50
50 inserted at end

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 9
List: 10->30->60->50->NULL

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 3
Insert new value: 35
Before node: 30
35 inserted before 30

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 4
Insert new value: 40
After node: 30
40 inserted after 30
```

```
1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 9
List: 10->35->30->40->60->50->NULL

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 8
Enter node to search: 60
Node 60 found at position 5

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 8
Enter node to search: 99
Node 99 not found

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 5
10 deleted from beginning

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 6
50 deleted from end

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 7
Enter node to delete: 35
Node 35 deleted

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 7
Enter node to delete: 99
Node 99 not found

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 9
List: 30->40->60->NULL

1.Insert at beginning  2.Insert at end  3.Insert before a node
4.Insert after a node  5.Delete from beginning  6.Delete from end
7.Delete a node  8.Search  9.Display  10.Exit
Enter choice: 10
```

Question 2: Count Occurrences of a Key and Delete All of Them

Input: 1->2->1->2->1->3->1, key: 1 Output: Count: 4, Updated Linked List: 2->2->3

Program:

```
/* Count occurrences of a key and delete all of them */
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};
struct node *head = NULL, *last = NULL;

void insertEnd(int x) {
    struct node *p = (struct node*)malloc(sizeof(struct node));
    p->data = x;
    p->next = NULL;
    if (head == NULL) head = p;
    else last->next = p;
    last = p;
}

void display() {
    struct node *t = head;
    while (t != NULL) {
        printf("%d", t->data);
        if (t->next != NULL) printf("->");
        t = t->next;
    }
    printf("\n");
}

int main() {
    int n, i, x, key, count = 0;
    struct node *t, *prev = NULL;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter %d values: ", n);
    for (i = 0; i < n; i++) { scanf("%d", &x); insertEnd(x); }
    printf("Enter key: ");
    scanf("%d", &key);

    printf("Linked List : ");
    display();

    t = head;
    while (t != NULL) {
        if (t->data == key) {
            count++;
            if (prev == NULL) head = t->next;     /* deleting first node */
            else prev->next = t->next;
            free(t);
            if (prev == NULL) t = head;
            else t = prev->next;
        } else {
            prev = t;
            t = t->next;
        }
    }

    printf("Count: %d\n", count);
    printf("Updated Linked List: ");
    display();
    return 0;
```

```
}
```

Output:

```
Enter number of nodes: 7
Enter 7 values: 1 2 1 2 1 3 1
Enter key: 1
Linked List : 1->2->1->2->1->3->1
Count: 4
Updated Linked List: 2->2->3
```

Question 3: Find the Middle of a Linked List

Input: 1->2->3->4->5 Output: 3

Program:

```
/* Find the middle of a linked list (slow and fast pointer) */
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};
struct node *head = NULL, *last = NULL;

void insertEnd(int x) {
    struct node *p = (struct node*)malloc(sizeof(struct node));
    p->data = x;
    p->next = NULL;
    if (head == NULL) head = p;
    else last->next = p;
    last = p;
}

int main() {
    int n, i, x;
    struct node *slow, *fast, *t;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter %d values: ", n);
    for (i = 0; i < n; i++) { scanf("%d", &x); insertEnd(x); }

    printf("Linked List : ");
    for (t = head; t != NULL; t = t->next) printf("%d->", t->data);
    printf("NULL\n");

    slow = head;
    fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;          /* moves 1 step  */
        fast = fast->next->next;    /* moves 2 steps */
    }
    printf("Middle: %d\n", slow->data);
    return 0;
}
```

Output:

```
Enter number of nodes: 5
Enter 5 values: 1 2 3 4 5
Linked List : 1->2->3->4->5->NULL
Middle: 3
```

Question 4: Reverse a Linked List

Input: 1->2->3->4->NULL Output: 4->3->2->1->NULL

Program:

```
/* Reverse a linked list */
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};
struct node *head = NULL, *last = NULL;

void insertEnd(int x) {
    struct node *p = (struct node*)malloc(sizeof(struct node));
    p->data = x;
    p->next = NULL;
    if (head == NULL) head = p;
    else last->next = p;
    last = p;
}

void display() {
    struct node *t;
    for (t = head; t != NULL; t = t->next) printf("%d->", t->data);
    printf("NULL\n");
}

int main() {
    int n, i, x;
    struct node *prev = NULL, *curr, *next;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter %d values: ", n);
    for (i = 0; i < n; i++) { scanf("%d", &x); insertEnd(x); }

    printf("Original List: ");
    display();

    curr = head;
    while (curr != NULL) {
        next = curr->next;      /* save next node          */
        curr->next = prev;      /* reverse the link        */
        prev = curr;            /* move prev one step      */
        curr = next;            /* move curr one step      */
    }
    head = prev;

    printf("Reversed List: ");
    display();
    return 0;
}
```

Output:

```
Enter number of nodes: 4
Enter 4 values: 1 2 3 4
Original List: 1->2->3->4->NULL
Reversed List: 4->3->2->1->NULL
```